

6/1 – Team Intro & Product Overview

Team Role

- Their job is to represent the customer
- Responsible for determining who the customers are and what problems they have
- Build products that are pervasive in the market and that customers are willing to pay for

Product Side Responsibilities

- **Establish strategic direction** – define the vision for the product
- **Tactical execution** – get it built correctly
- **Connect the dots** – communicate and empower the broader organization

UX Side Responsibilities

- **Discovery** – answer questions, uncover opportunity, and vet ideas
- **Design** – determine how people engage with it, what it looks like, and how it works
- **Validation** – determine what customers think, if we succeeded, and what comes next

Project: Social Engagement Accelerator (SEA)

- Aggregates student social data from across campus (clubs, events, dining, housing, etc) into one dashboard
- Applies an engagement score to each student, group, event, and institution
- Helps staff identify socially isolated or at-risk students early
- Enables intervention – connecting students to counselors, buddies, or resources
- Serves advisors, group/event owners, and institutional leadership with role-specific views

Understanding Evisions

- Helps higher education organizations work better – software eases administrative load so staff can focus on learning and discovery
- Exclusively a higher ed company (25+ years, 900+ institutions)
- Builds data reporting, document management, and payment tools that sit on top of whatever systems a school already uses
- Known for their helpdesk – widely regarded as the best support among higher ed software vendors
- Clients describe support as exceptional, 5-star, and unlike any other vendor
- Helpdesk email: helpdesk@evisions.com
- Support + Community portal at support.evisions.com – users share templates, DataBlocks, and resources via a "CO-OP"

Solutions

- **Argos** – flagship enterprise reporting tool; lets faculty and staff pull reports and build dashboards without needing IT; works across on-premise and SaaS environments
- **Argos X** – modern evolution of Argos with updated UI and more advanced analytics capabilities
- **Data Agent** – AI-powered natural language assistant built into Argos X; lets users ask questions in plain English and get answers grounded in trusted, governed data; shows logic behind every response (not a black box)
- **Academic Success Accelerator** – blends LMS and SIS data into one view so advisors can spot student disengagement weeks earlier, before it shows up in grades or withdrawal reports

- **FormFusion** – automates creation and delivery of documents, forms, and letters; brings document production in-house and cuts printing/postage costs
- **IntelleCheck** – payment and check processing tool; institutions use it to go paperless on disbursements (one college saved \$100K/year)

Key Info & Lingo

- **SIS (Student Information System)** – core database of records for a university; stores enrollment, grades, financial aid, demographics, degree progress, etc; the "spine" of all student data at an institution
- **Ellucian** – dominant SIS/ERP provider in higher ed; powers roughly 3,000 institutions serving 21 million+ students across 50 countries; makes Banner and Colleague
- **Banner** – Ellucian's ERP platform widely used by large universities with complex reporting, compliance, and scalability requirements; Evisions' tools connect directly to Banner (both on-premise and SaaS)
- **Colleague** – Ellucian platform well-suited for institutions that value flexibility and faster system updates; more common at mid-sized and community colleges; Evisions fully integrates with Colleague
- **Banner/Colleague SaaS Starter Packs** – pre-built DataBlocks (report templates) deployable immediately in Banner or Colleague SaaS environments; cover admissions, enrollment, finance, HR, and instruction; remove need to build reports from scratch
- **LMS (Learning Management System)** – where course content, assignments, and student activity live (Canvas, Blackboard, Moodle, D2L, etc); separate from the SIS; the Academic Success Accelerator's value prop is bridging these two systems
- **Accelerators** – Evisions' term for pre-built, focused analytics products that solve a specific institutional problem fast
- **Advancement** – higher ed term for fundraising, alumni relations, and donor management; Banner Advancement manages donor info and connects alumni to development officers
- **Black Box vs Transparency** – a "black box" system gives an answer but hides how it got there; Data Agent is the opposite – every answer is grounded in trusted data, respects existing permissions, and shows the logic behind the response; critical in higher ed where advisors need to defend interventions

Research & The Higher Ed Market

- **Core Principle** – Step 1 of software development is finding the right opportunity; everything else (roadmap, design, build, launch) depends on getting this right; get the opportunity wrong and the best engineering builds the wrong thing
- **Framework: Software** – Who's the market/segment? Do they actually have the problem? Who's the competitor/incumbent? What features matter to this buyer? Is there a real budget and return on it?
- **Segmenting Higher Ed** – no single "higher ed" exists; a community college and an Ivy share almost nothing operationally; segmentation dimensions: level (2yr/4yr), control (public/private), Carnegie class, size, population (HBCU/HSI), academic focus, residency, affiliations
- **SEA segmentation dimensions that matter most:**
 - Residential status – residential campuses have engagement everywhere (job = measure/steer it); commuter campuses barely have it (job = create it); these are fundamentally different products
 - Size – belonging at 1,500 looks nothing like belonging at 30,000; large schools fight

anonymity, small ones know everyone

- **The Two-Institution Contrast:**

- Institution A (4-yr private residential R1, ~12,200 students) – clubs, Greek life, athletics, packed events; engagement exists, needs steering; this is the SEA’s target
- Institution B (2-yr public commuter, ~14,100 students) – students arrive, attend, leave for work/family; engagement happens in 15-min windows; different product entirely

- **How Research Gets Done:**

- Primary – interviews, surveys, focus groups; you go get it directly from real users
- Secondary – existing news, association data, research centers (Inside Higher Ed, Chronicle, Higher Ed Dive, EDUCAUSE, NACADA, AACRAO, NASFAA, niss.gsu.edu)

- Good research triangulates – a claim is strongest when an interview and a published source point the same way

6/2 – Grooming Meeting

What Is a Grooming Meeting

- A meeting where the product team takes tickets (feature requests, bug reports, enhancements) from internal teams or customers and works through what it would actually take to implement them
- Output: tickets get scoped, documented, and handed off with clear marching orders to UX or engineering

Ticket 1 – Remove Feature Flag for Data Masking

- Data masking was previously hidden behind a feature flag (only shown to select users)
- Now that it is ready for production, the flag needs to be removed so all customers can access it automatically
- Engineers know what to do – ticket moved to input queue, grooming label removed
- Flow after grooming: engineering builds it → QA tests it → confirmed → closed

Ticket 2 – Deletion Confirmation for Deleting Masks

- When a user deletes a masking rule, there is currently no confirmation or undo
- Request: add some form of confirmation so users do not accidentally lose work
- Discussion landed on two patterns depending on action:
 - Single item deletion – toast notification with an undo option
 - Bulk/multiple item deletion – modal (confirmation dialog: “are you sure you want to delete X items?”)
- Bulk delete also requires checkboxes per item plus a select all option
- Data blocks may need separate handling from column names and values
- Ticket passed to UX to create mockups before going to engineering

Ticket 3 – Thumbs Up/Thumbs Down on Data Agent Responses

- Request: add feedback buttons to the AI chat so users can rate responses
- Bigger question the team worked through: what do we actually do with that data
- Key decisions/discussion:
 - Feedback does not go back to the underlying AI model – it stays with Evisions for internal use

- Want to tie sentiment to context (what data block was being used, what topic the question was about)
- Data security concern: cannot store raw questions because they may contain student SSNs or sensitive info
- Solution: use AI to generate a safe one-sentence summary of the question/response instead of storing raw text
- Start small – store data block name plus a one/two word topic tag plus thumbs up/down, then iterate
- Optional: add a "more feedback" link that routes users to the support portal
- The eventual goal is to build prebuilt prompts that reduce token usage for clients
- Personal question from chat: "If the user gets stuck in a loop with the chat, could there be an easier way out?"
- Answer: start a new chat to reset context; also adding a "more feedback" link to the support portal addresses this

Key Info & Lingo

- **Ticket** – a documented request in a project management system (like Jira) for a feature, bug fix, or enhancement; contains a title, description, context, screenshots, and scope details; moves through stages: grooming → UX → input queue → in progress → QA review → done
- **Agile** – a software development methodology built around iterative, incremental work rather than building everything at once; teams work in cycles (sprints), continuously delivering and improving; embraces change
- **Waterfall Method** – the opposite of agile; a linear, sequential process where each phase (requirements → design → build → test → deploy) must be fully completed before the next begins; once you move forward, going back is costly; common in older enterprise software
- **Kanban Board** – a visual board used to track work as it moves through different stages; Evisions uses Jira's Kanban board with columns like Input Queue, In Progress, QA Under Review, Done; each ticket lives in one column at a time
- **Swim Lane** – a horizontal section of a Kanban board dedicated to a specific team or workflow; UX had its own swim lane – when a ticket needed mockups, it was moved into the UX lane
- **Jira** – Atlassian's ticket tracking and project management tool; the system Evisions uses to create, track, and manage all development work
- **Data Agent** – Evisions' AI-powered chat assistant built into Argos X; lets users ask questions in plain English against their institution's data; responses are grounded in governed data, not a black box
- **Data Masking** – a settings feature within Data Agent that lets institutions choose which data blocks (or specific columns/values within them) to exclude from the AI's access; protects sensitive data; acts as a privacy filter between the AI and raw institutional data
- **Feature Flag** – a toggle in the code that lets developers show or hide a feature for specific users or institutions without redeploying the whole app; used to safely release features to a subset of users first; Evisions uses PostHog to manage feature flags and track usage metrics
- **Toast Notification** – a small, temporary pop-up that appears briefly on screen and then disappears automatically; less intrusive than a modal; used for low-stakes confirmations like "item deleted – undo?"
- **Modal** – a pop-up dialog that requires the user to actively interact with it before continuing (eg clicking "confirm" or "cancel"); more intrusive than a toast; used for higher-stakes actions like bulk deleting

- **SQL + Query** – SQL (Structured Query Language) is the language used to retrieve data from a database; a query is a specific request that says "go find me this data"; DataBlocks contain the SQL queries that pull data from a school's SIS or other systems
- **Regex (Regular Expression)** – a pattern-matching syntax used to identify specific strings of text in data; used in data masking to automatically flag and hide fields matching certain patterns (eg any column containing "SSN")
- **PICL (Product Introduction Checklist)** – a structured framework for planning, executing, and tracking a product launch or feature release; used for new products, new features, or significant functionality changes; reduces oversight and coordinates across all departments
- **Cloud vs On-Prem Release Context:**
 - Argos X is cloud-based – Evisions updates the code on their end and customers get it automatically on refresh
 - Maps/Argos (legacy) are on-premise – customers must download and install new versions like traditional software

PICL Phases

- **Phase 1 – Idea** (owned by Product): initial business case; product vision; roadmap (3, 6, 9, 12 months); themes and epics identified
- **Phase 2 – Inception** (owned by Product): positioning statement and value prop; themes/epics refined; telemetry metrics determined via PostHog; release kickoff sent to internal stakeholders; wireframes and UI requirements drafted
- **Phase 3 – Elaboration** (owned by Product + Engineering + Marketing): use cases confirmed; beta sites determined; localization requirements assessed; A/B concept testing; release schedule planned; engineering drafts product quality plan; marketing begins building webinars, blog posts, and collateral
- **Phase 4 – Construction & Launch** (owned by Product + Engineering + Marketing + Support + Sales): beta program managed; product introduction presentation created; internal trainings run; release readiness check across all departments; engineering QA and security checks; feature flags enabled for internal then production; launch day: marketing posts, community announcements, web help, production builds, emails, partner notifications all go live
- **Phase 5 – Post-Launch** (owned by Product + Marketing + Professional Services): track KPIs (downloads, installs, active usage, feature adoption); feedback loop with Customer Engagement Team; hotfix/unplanned release if severely broken (kills timeline for next project, avoided at all costs); retrospective; accessibility updates (VPAT form + remediation form); conference presentations; professional services update training documentation; process repeats for next release immediately

Data Sourcing

- **The Core Problem** – Evisions does not generate its own data; it must connect to data that already exists at institutions; no single "student social life" database exists on any campus – every data point lives in a different system owned by a different office
- **How to Think About It** – users come to the application with a specific question; the question determines what data you need; doctor analogy: do not run every test, only the ones that answer the patient's problem; only surface data that answers the question, nothing superfluous
- **The Fragmentation Reality** – dining swipes, club rosters, building access, athletics, career services, res life, parking – all separate systems, often 15+ years old, that do not talk to each

other; getting data is not just a tech problem, it is a people problem; each office has to agree to share

- **Privacy Constraints:**
 - Health data – HIPAA-protected, cannot be shared across campus
 - Student records – FERPA governed
 - Off-campus activity – institutions simply do not have it
- **The "Too Much Ask" Problem** – if answering one question requires connecting to 15 systems, institutions will walk away; every data decision shapes what the product can actually build, you can only deliver what the data supports

6/3 – Roadmapping & UX

What a Roadmap Is

- Strategic sequencing of features with timing and rationale
- Can be organized by release, product/feature, functional area, technical area, or business goals

Roadmap Planning Constraints

- MVP = smallest shippable value customers will pay for
- Dependencies – foundational features must precede dependent ones
- Time/Cost/Scope triangle – fix two, the third is determined for you
- Hard deadline: September 1 launch (miss it = wait a full year)
- Example constraint: 12 features at 1 week each; only 9 weeks of capacity, so 3 go to the parking lot

SDLC (Software Development Life Cycle)

- **Step 1 – Find the opportunity** – identify a real market problem worth solving; determine who has it, how badly, and whether there is a viable business in addressing it
- **Step 2 – Build the roadmap** – sequence and prioritize features given constraints (time, cost, scope); decide what is in the MVP and what is parked
- **Step 3 – Design the solution** – translate requirements into wireframes and prototypes (eg Figma); define what the product looks like and how users move through it
- **Step 4 – Build the product** – engineering constructs the features per the user stories and designs
- **Step 5 – Launch/release** – ship the product to customers; in SaaS this is often a staged rollout, not a single moment
- **Step 6 – Monitor & react** – track usage, engagement, and feedback post-launch; use data to decide what to fix, improve, or build next

Feature Documentation

- Written as user stories: "As a [role], I want [goal], so that [benefit]"
- Includes acceptance criteria to define "done"
- Keeps contributors creative rather than micromanaged

Parking Lot

- Features that do not make the MVP window are formally deferred, not dropped

UX Fundamentals

- UX = how a person feels interacting with a product/system/service; UI = the visual elements inside it
- Core goal: make digital products easy to use (useful, easy, efficient)
- Always design for the least technical user ("design for grandma")
- Take your ego out – it is never about your preferences, it is about the user

Design Thinking Stages

- Discovery → Research → Ideation → Testing/Prototyping → Iteration (never truly "done")
- Human-centered, iterative, collaborative, bias toward action
- MVP = minimum viable product; smallest feature set that lets real users test core functionality

Key UX Terms

- **UX Persona** – person using the product; **Buyer Persona** – person approving the purchase (often different)
- **Use Cases** – specific problems a user is trying to solve
- **Wireframe** – black-and-white layout sketch (blueprint), no color or final copy
- **Global Filter** – affects all dashboard charts; **Local Filter** – affects only one chart
- **Drill-Down** – clicking a chart element to see the underlying student list
- **Modal/Drawer** – popup or slide-in panel for detail without navigating away
- **Tooltip** – hover popup showing a data value on a chart
- **Cognitive Load** – mental effort required; good UX reduces it
- **Data Story** – charts plus narrative context telling a "why," not just "what"
- **Thematic Analysis** – tagging transcript/survey segments into recurring themes (done in Dovetail)
- **UX Audit** – structured heuristic review of an existing product, scored against usability standards

Gestalt Principles

- **Similarity** – elements that look alike are perceived as related
- **Continuity** – eyes follow the smoothest path (sliders, carousels, scroll)
- **Closure** – the brain fills in missing info to complete a shape (FedEx arrow, loading circles)
- **Proximity** – elements close together are seen as related; most practical for dashboard layout
- **Prägnanz** – people simplify complex visuals automatically; do that work for them
- **Common Fate** – objects moving/pointing the same direction are perceived as a group

Nielsen Norman Heuristics

- Visibility of system status
- Match between system and real world
- User control and freedom
- Consistency and standards
- Error prevention
- Recognition over recall
- Flexibility and efficiency
- Aesthetic and minimalist design
- Help users recognize and recover from errors
- Help and documentation

SUS Scores (System Usability Scale)

- 10-question Likert survey, alternating positive/negative, produces a 0–100 score
- 80+ = good, ship it; 70–79 = needs work; below 70 = do not release
- Use as a baseline over time, not a one-time verdict
- Always ends with open-ended "what would you change"

Wireframing

- Purpose: get ideas on paper fast for testing, not to produce a finished design
- Apply gestalt: proximity for groupings, similarity for consistency, prägnanz for simplicity
- For every chart: know (1) what data it shows, (2) who owns it on campus, (3) what the field/column name is
- Dashboard = overview; Pages = detail; decide what triggers a modal vs a new page

6/4 – Weekly Review Meeting

Key Takeaways

- Covered the full software development life cycle from the product angle: market segmentation, product management, and UX/design
- Wireframing activity brought the week's concepts together visually
- Big insight: people who build the product are not the people who buy it (always calibrate to your audience)
- Attending a real internal business meeting was a highlight – seeing cross-department collaboration in action
- More hands-on activities and group work preferred over back-to-back lectures

Coming Up

- Next week: engineering team, dashboard building, and potentially AI-assisted code development
- Capstone presentation in week 8 – take notes throughout every department rotation

Friday Format

- Jim Farrell (Chief Solutions Officer) on product
- Stuart on communications
- Donut-making class
- Connections session

Assessment Structure

- Teresa 50%, peer 30%, self 20%
- Graded on professionalism, communication, collaboration, initiative, and growth mindset
- Digital portfolio encouraged if time allows (website-style resume with project reflections)

6/5 – Business & Communications Seminars

Business Seminar: How a B2B Software Company Functions

How the Business Works

- Value flows across functions – Market, Sales, Product, Engineering, Services, Support, and Success – with the Executive Team providing direction and Finance providing control

- No single team creates value alone; it happens through handoffs, feedback loops, and shared accountability

How Product Decisions Get Made

- Ideas get filtered through four lenses – customer feedback, market signal, business impact, and feasibility – all running through a product strategy funnel
- The hardest part is not generating ideas, it is choosing what not to build

How Leaders Make Decisions

- Business is a series of tradeoffs with no perfect answer – build now vs later, speed vs quality, revenue today vs strategy tomorrow
- Leadership is about making the best call with incomplete information, then learning quickly

Advice for Early-Career Professionals

- Ask better questions before jumping to answers
- Write clearly so your thinking travels
- Learn the customer
- Support opinions with data
- Communicate risks early – no surprises is a professional superpower
- Own the follow-through, because reliability compounds
- You do not need all the answers, you need to become someone others trust with the next problem

Communications Seminar: Conversation as a Professional Skill

Core Argument

- A good conversation is how you get remembered – in an internship, an interview, and on the call that lands the job
- Without it you leave a forgettable impression; with it people open up and remember your name

What Makes Conversation Work

- Genuine curiosity and asking real follow-up questions rather than waiting for your turn
- Making the other person feel like their answers are interesting rather than being evaluated
- Pace matters – comfort with silence signals confidence and reduces pressure on the other person

Why Conversation Stalls

- Closed questions, talking too much about yourself without turning it back, and trying to impress rather than connect
- The harder you try to seem impressive, the more the other person pulls back

Why It Compounds

- People refer, advocate, and open doors for people they actually know
- Connections made early – especially with senior people – can matter enormously years later
- Knowing people across functions accelerates how fast you understand a business, which ties directly back to the business seminar's point that value is created through cross-functional handoffs, not within any single team